

System Audit & Reliability Report

CoWork Multi-Tenant Booking API Platform

Prepared by Team: du_neurons

IUT 12th ICT Fest — Agentic AI Hackathon Preliminary Round

July 9, 2026

Abstract

This report provides a formal and detailed breakdown of fourteen (14) logical defects, data validation issues, performance bottlenecks, and concurrency vulnerabilities discovered and mitigated in the CoWork REST API. All patches have been validated against the system's specifications, multi-tenancy requirements, and concurrent transaction execution boundaries.

1 Executive Summary

During the preliminary round, the CoWork coworking space booking platform was subjected to a rigorous system audit. Fourteen core issues spanning API contract compliance, multi-tenant isolation, cache invalidation freshness, database query latency, and transaction race conditions were identified and patched. All fixes preserve the original API signature while securing the platform against concurrent race attacks and database lock starvation.

2 Summary of Identified Issues

ID	Vulnerability / Issue Title	Component / Location	Severity
01	Error Code Formatting (Spaces vs. Underscores)	app/errors.py	High
02	Missing Validation on Room Capacity & Rate	app/schemas.py	Medium
03	Datetime UTC Format Designator ('+00:00' vs 'Z')	app/timeutils.py	High
04	Registration Org-Creation Concurrency Race	app/routers/auth.py	High
05	Booking Overlap/Quota Race Condition (TOCTOU)	app/routers/bookings.py	Critical
06	Incomplete and Stale Caching Integration	app/routers/rooms.py, admin.py	Medium
07	Export CSV Header Formatting Deviation	app/services/export.py	High
08	Missing Date Range Validity in Usage Report	app/routers/admin.py	Medium
09	$O(N)$ $N+1$ Database Queries in Usage Report	app/routers/admin.py	High
10	SQLite default DELETE mode write lock bottleneck	app/database.py	Critical
11	Unbounded Cache Memory Exhaustion (Cache DoS)	app/cache.py	High
12	Cross-Day Booking Availability Inconsistency	app/routers/rooms.py, bookings.py	High
13	Booking Ref-Code Concurrency Error Mapping	app/routers/bookings.py	Medium
14	In-Memory Token Revocation Persistence Loss	app/auth.py, models.py	High

3 Detailed Audit & Resolution Log

Bug 01: Error Response Code Formats (Spaces vs. Underscores)

Location: `app/errors.py`

Problem: Raised system exception codes utilized Pythonic snake_case underscores (e.g. `ROOM.CONFLICT`), but the specification mandates space-separated capitals (e.g. `ROOM CONFLICT`).

Fix: Updated `app_error_handler` exception handler to dynamically map underscores to spaces globally before JSON serialization, preserving clean exception names internally.

Bug 02: Missing Validation for Room Capacity and Rates

Location: `app/schemas.py` (lines 21–25)

Problem: Lack of lower-bound constraints allowed creation of rooms with negative or zero capacities and rates.

Fix: Bound parameters in `RoomCreateRequest` using Pydantic's `Field(gt=0)` constraint.

Bug 03: UTC Format Designator in Datetime Serialization

Location: `app/timeutils.py` (lines 19–26)

Problem: Response datetimes returned Python's default `+00:00` string designator for UTC offsets instead of the explicit ISO 8601 designator `Z`.

Fix: Modified the `iso_utc()` formatter to replace timezone offset suffixes with `Z`.

Bug 04: Concurrency Race Condition in Registration

Location: `app/routers/auth.py` (lines 27–35)

Problem: Concurrently registering users for a new organization could cause organization duplicated uniqueness checks to evaluate to false in parallel threads, resulting in a crashed commit and a false registration failure.

Fix: Placed a database transaction lock `BEGIN IMMEDIATE` at the initiation of the registration method to serialize organization registration check-and-creates.

Bug 05: Concurrency Race Conditions (TOCTOU) in Booking Creation/Cancellation

Location: `app/routers/bookings.py` (lines 85–145)

Problem: Overlap and quota checks occurred inside standard deferred transactions, meaning concurrent overlapping bookings or quota bypasses were not serialized at the database engine.

Fix: Enforced `db.execute(text("BEGIN IMMEDIATE"))` at transaction start inside both booking creation and cancellation blocks to serialize all mutations.

Bug 06: Incomplete and Stale Caching Integration

Location: `app/routers/rooms.py` and `app/routers/admin.py`

Problem: In-memory caches were ignored in GET paths. Furthermore, creating rooms left reports cached with stale data.

Fix: Integrated cache lookup-and-writes in the availability and report paths, and invalidated the organization's report cache upon room creation.

Bug 07: Export CSV Header Spacing Deviations

Location: `app/services/export.py` (lines 11–20)

Problem: CSV export column headers used underscores (e.g. `price_cents`) instead of the exact specification header titles (e.g. `price cents`).

Fix: Modified `EXPORT_HEADER` to align strictly with the target specification.

Bug 08: Missing Date Range Validation in Usage Report

Location: `app/routers/admin.py` (lines 17–29)

Problem: Usage reports accepted start dates succeeding end dates, returning empty lists instead of validation errors.

Fix: Added validation verifying that `from_date ≤ to_date`, returning `INVALID_BOOKING_WINDOW` (400) otherwise.

Bug 09: N+1 Database Query Performance in Usage Report

Location: `app/routers/admin.py` (lines 41–65)

Problem: Usage report looped through all organization rooms and executed a separate query per room to count bookings, resulting in $N + 1$ database calls.

Fix: Optimized the aggregation logic to perform a single query using an `outerjoin` and `group_by` at the database level.

Bug 10: SQLite Default DELETE Journal Mode (Concurrency Bottleneck)

Location: `app/database.py` (lines 13–17)

Problem: Default SQLite configuration locked database files for writes, blocking concurrent readers and resulting in connection timeouts.

Fix: Enabled Write-Ahead Logging (`PRAGMA journal_mode=WAL`) and configured normal synchronization (`PRAGMA synchronous=NORMAL`) to support safe concurrent reads and writes.

Bug 11: Unbounded Cache Memory Exhaustion (Cache DoS)

Location: `app/cache.py`

Problem: The response caching structures used raw Python dicts without size constraints or TTL, allowing attackers to exhaust memory (DoS) by querying infinite unique combinations.

Fix: Wrapped caching collections in a thread-safe `LRUCache` limiting max capacity to 1000 entries.

Bug 12: Cross-Day Booking Availability Inconsistency (API Leak)

Location: `app/routers/rooms.py` and `app/routers/bookings.py`

Problem: Availability queries checked only start times on the target day. Cross-day bookings ending on that day were omitted in availability but blocked bookings with a `409 ROOM CONFLICT`. Invalidation also missed the end dates.

Fix: Modified checks to detect overlapping bookings (`start_time < day_end` and `end_time > day_start`). Cache invalidation now clears both dates.

Bug 13: Concurrency Reference Code Collision Error Mapping (Error Masquerade)

Location: app/routers/bookings.py

Problem: The API creation routine caught generic database `IntegrityErrors` on transaction commit (e.g. from unique reference code conflicts) and incorrectly mapped them to `409 ROOM CONFLICT`.

Fix: Created a 3-attempt database write retry loop that generates a new code upon collision, raising a `500 INTERNAL_ERROR` if all fail.

Bug 14: In-Memory Token Revocation Persistence Loss

Location: app/auth.py, app/models.py, and app/routers/auth.py

Problem: Logout blacklists and single-use refresh token tracking lists were held purely in-memory. Server restarts or multi-process instances would lose this state, enabling token replay attacks.

Fix: Created database-persisted tables `revoked_tokens` and `used_refresh_tokens` to log claims persistently.